

HandsOn - 3

Simone Passèra

Jan 08, 2025

1 Introduction

The objective of this hands-on is to play with Dynamic Programming in Rust. There are two problems to solve.

2 Code Implementation

2.1 Problem 1

I defined the subproblems using an array `table`, where `table[d]` represents the maximum number of attractions that can be visited using exactly `d` days. For each city, I computed a `prefix_sum` array, which contains the total number of attractions that can be visited by spending up to a certain number of consecutive days in that city.

For each city and for each number of available days `d`, I considered all possible choices of `k` days to spend in the current city (where $1 \leq k \leq \min(\text{city.len}(), d)$). I filled the table from right to left, using the recurrence:

$$\text{table}[d] = \begin{cases} 0 & \text{if } d = 0, \\ \max_{1 \leq k \leq \min(\text{city.len}(), d)} (\text{table}[d - k] + \text{prefix_sum}[k]) & \text{if } d > 0. \end{cases}$$

At the end of the iteration over all cities, the value of `table[days]` represents the maximum number of attractions that can be visited using all the available days.

The time complexity of the solution is $O(nD^2)$, where `n` is the number of cities and `D` is the total number of days. This comes from iterating over all cities, all days, and all possible days spent in the current city, which can be at most `D` for each iteration.

2.2 Problem 2

To solve the problem, I used an approach based on the Longest Increasing Subsequence (LIS), optimized with binary search.

First, I sorted the topics by increasing beauty (and by increasing difficulty in case of ties). Second, I focused on finding the longest increasing subsequence (LIS) of the difficulties of the sorted topics. Each element in this subsequence represents a topic that can be included in the course.

To find the LIS, I applied the algorithm that we have seen in class. For each difficulty, I searched for its correct position in the `table` list, which maintains the current LIS*, using binary search. If the difficulty was greater than all the elements in `table`, I added it to the list. Otherwise, I replaced the first element in `table` that was greater than or equal to the current difficulty, ensuring that the list remains compact and ready for further expansion.

Finally, the length of the table list at the end of the process represents the maximum number of topics that satisfy the required constraints. The time complexity is $O(n\log(n))$.